Credit Risk Scoring & Loan Default Analysis

Machine-Learning Backend: Architecture and Step-by-Step Guide

Member 4 - Backend Machine Learning

Item	Details
Purpose	Learn from historical loans, predict the probability that a new borrower defaults, and explain why.
Stack	Python 3.12, FastAPI, Pydantic, pandas, NumPy, scikit-learn, SHAP, joblib, pytest
Models	Logistic Regression, Random Forest, Decision Tree (compared automatically)
Interface	8 REST endpoints returning JSON, consumed by the frontend / dashboard team
Location	Everything lives in the **backend/** folder
Generated	23 October 2026

Contents

1 The big picture

2 Folder structure

3 System architecture

4 The ML pipeline, step by step

5 Inside the saved model pipeline

6 Prediction and explanation flow

7 API reference

8 Error handling

9 Current results

10 Run it yourself, step by step

11 Testing

12 Limitations and next steps

1. The big picture

This backend is the "brain" of the project. It has no user interface. Other team members' frontend and dashboard send it HTTP requests and receive JSON answers. Its job has four parts:

1. **Understand the data** - load a loan CSV, check its quality, and report problems.
2. **Learn** - train three classification models on past loans and keep the best one.
3. **Predict** - for a new borrower, return the probability of default, a yes/no class and a Low/Medium/High risk band.
4. **Explain** - use SHAP to show which borrower details pushed the risk up or down.

Three principles shaped every design decision:

- **No cheating (leakage prevention).** Information only known after a loan ends, such as recovered amounts, is removed. The test data is never used for learning or choosing.
- **Honest numbers.** Every metric is calculated from real predictions on held-out data. Nothing is hardcoded.
- **Safe inputs.** Every request is validated; bad data returns a clear error that names the field at fault.

2. Folder structure (all inside backend/)

Path	What it contains
app/main.py	Creates the FastAPI app, CORS, error handlers, the health endpoint, and registers all routes.
app/config.py	Settings (folders, 20 MB upload limit, CORS origins), default risk bands, disclaimer texts.
app/routes/	Thin HTTP layer: data.py, training.py, prediction.py, explanation.py, deps.py. Parse the request, call a service, return JSON.
app/schemas/credit_risk.py	Pydantic models describing every request and response (the API contract).
app/services/data_processing.py	CSV loading, target mapping, feature typing, leakage detection, quality report.
app/services/model_training.py	Train/val/test split, preprocessing pipeline, the 3 models, selection, joblib save/load.
app/services/evaluation.py	Precision, recall, F1, ROC-AUC, confusion matrix, classification report, threshold tuning.
app/services/prediction.py	Validates borrower records, scores them, assigns risk bands.
app/services/explainability.py	SHAP explainers, global importance, per-borrower explanations.
app/services/store.py	Keeps the current dataset and model in memory and on disk; blocks parallel training.
app/services/errors.py	Named error types, each mapped to an HTTP status code.
data/synthetic_loans.csv	SYNTHETIC development dataset (3,030 rows). Uploaded files go to data/uploads/.
models/	credit_risk_model.joblib (trained pipeline) and metrics.json (all results).
scripts/	generate_synthetic_data.py and smoke_test.py (end-to-end check).
tests/	47 pytest tests: data, pipeline, metrics, prediction, SHAP, API, errors.
docs/	This PDF and the script that builds it.
README.md	Setup commands and the full API contract with examples.

3. System architecture

The code is split into layers. Each layer only talks to the one below it, so the HTTP details, the machine-learning logic and the storage can each change without breaking the others.

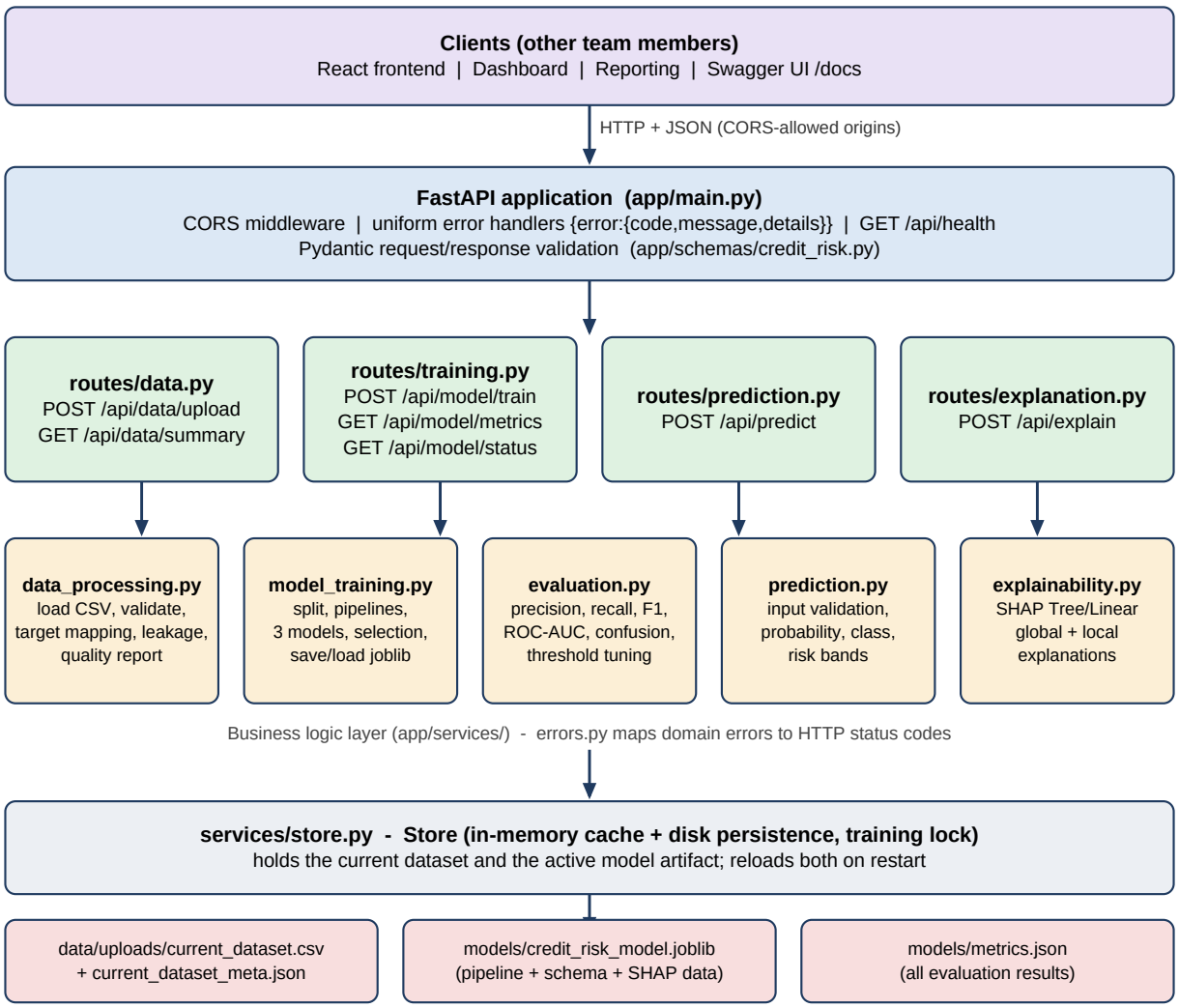
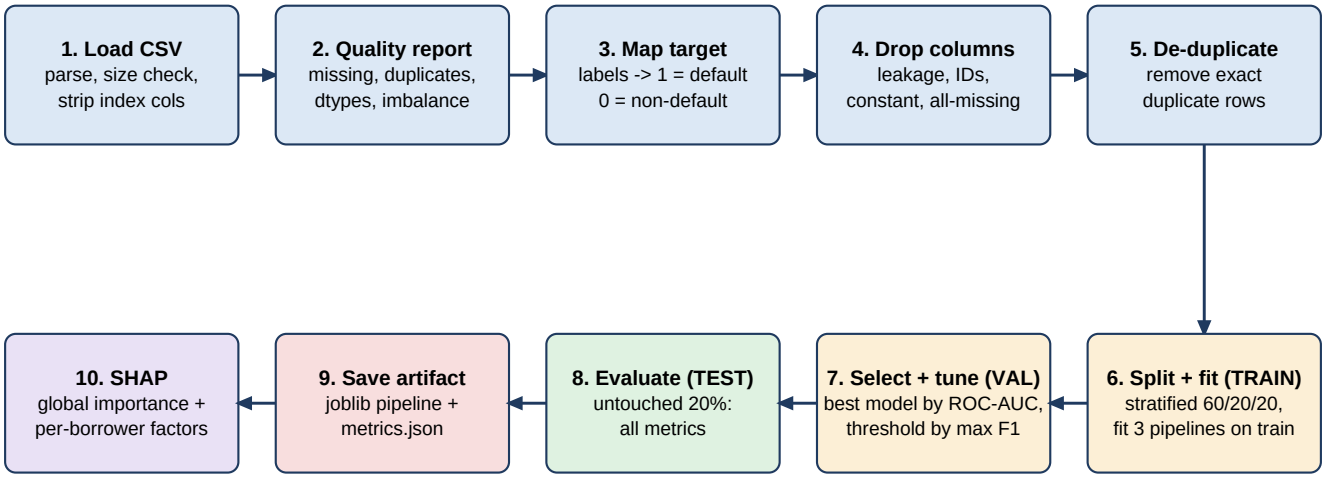


Figure 1 - Layered architecture of the ML backend

Layer	Responsibility	Why it is separate
Clients	Frontend, dashboard, reports (other members)	They only need the JSON contract, not Python.
FastAPI app	HTTP, CORS, validation, one error format	Consistent behaviour for every endpoint.
Routes	Translate HTTP into service calls	Kept thin so logic is testable without a server.
Services	All data and ML logic	Pure Python, reused by API, scripts and tests.
Store	Current dataset + model, persistence, training lock	Survives restarts; prevents two trainings at once.
Disk	CSV, joblib model, metrics JSON	No database existed in the project to integrate with.

4. The machine-learning pipeline, step by step



Preprocessing statistics are learned from the TRAIN split only; VAL picks the model; TEST is only used for the final report.

Figure 2 - What happens when POST /api/model/train is called

Step 1 - Load the CSV

The upload must be a .csv file under 20 MB. Empty files, unreadable text and duplicate column names are rejected. Index columns that pandas sometimes writes ("Unnamed: 0") are dropped.

Step 2 - Data-quality report

Counts missing values per column, exact duplicate rows, numbers stored as text (e.g. "38,000"), mixed types, and the class balance. It also suggests likely target columns. The report is returned on upload and by GET /api/data/summary.

Step 3 - Map the target

The target column and which labels mean "default" are configurable. Labels are normalised (case, spaces, 1/1.0/True). If the target has more than two values (for example "Current" loans whose outcome is unknown), you must list negative_labels so unknown outcomes are not silently counted as good loans. Rows with no target are dropped and counted.

Step 4 - Remove unusable or leaking columns

Dropped automatically: columns whose names signal post-outcome information (recoveries, total payments, last payment, settlement, outstanding principal...), ID columns, constant columns, all-missing columns and free text. A numeric column that on its own separates defaults almost perfectly (AUC 0.98 or more) is flagged as a warning. Extra columns can be excluded with exclude_columns.

Step 5 - Remove duplicates

Exact duplicate rows are removed before splitting, so the same loan cannot appear in both training and test data.

Step 6 - Split and train

Stratified, reproducible split: 60% train, 20% validation, 20% test, each with the same default rate. For each of the three models a full pipeline (Section 5) is fitted **on the training split only**. Because defaults are rare (about 20%), every model uses balanced class weights so it cannot score well by always saying "no default".

Step 7 - Pick the threshold and the model (validation split)

A model outputs a probability; a threshold turns it into yes/no. The threshold is tuned on validation data to maximise F1 for the default class, or to reach a minimum recall if min_default_recall is set. The model with the best validation score (ROC-AUC by default) is selected.

Step 8 - Evaluate (test split)

The untouched test split gives the final, honest numbers for every model: precision, recall, default-class recall, F1, accuracy, balanced accuracy, ROC-AUC, PR-AUC, Brier score, confusion matrix and the full classification report. Train ROC-AUC is reported too, so overfitting is visible (e.g. a much higher train than test score).

Step 9 - Save

The selected pipeline, the feature schema (allowed fields, training ranges, known categories), the threshold, risk bands, metrics and SHAP background data are saved to models/credit_risk_model.joblib. Metrics also go to models/metrics.json. The file is written to a temporary name first and then swapped in, so a crash cannot leave a half-written model.

Step 10 - SHAP check

Before the model is saved, global SHAP importance is computed once. If explanations fail, the model is not activated.

5. Inside the saved model pipeline

Preprocessing and the classifier are saved together as one scikit-learn Pipeline. This guarantees that a borrower is transformed at prediction time exactly as the training data was.

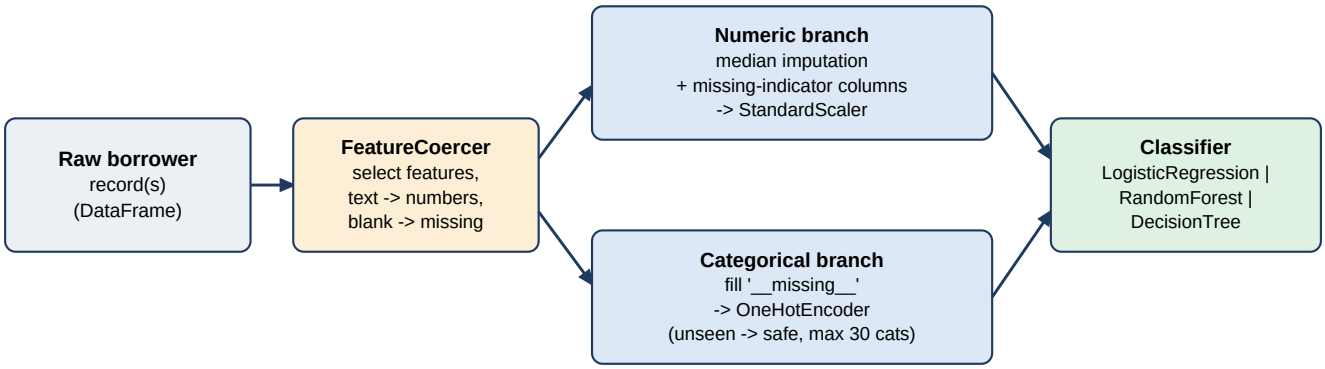


Figure 3 - The scikit-learn pipeline stored in the joblib file

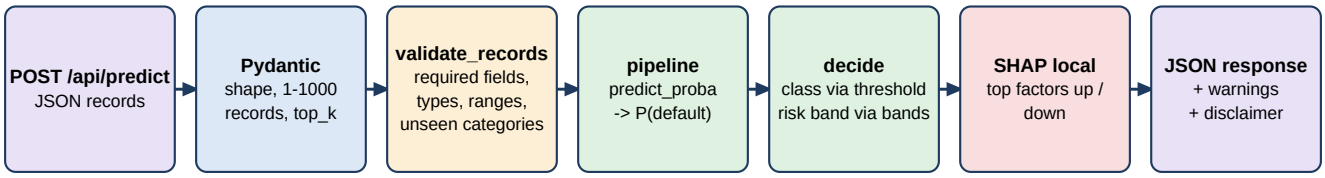
Component	What it does	Why
FeatureCoercer	Keeps only model features, converts "1,200" to 1200, blanks to missing, booleans to text.	Learns nothing, so it cannot leak information. Same cleaning at train and predict time.
Median imputation + indicators	Fills missing numbers with the training median and adds a "was missing" column.	Missingness itself can carry risk signal.
StandardScaler	Puts numeric features on a common scale.	Needed by logistic regression; harmless for trees.
'__missing__' + OneHotEncoder	Turns categories into 0/1 columns; rare ones (under 5 rows) are grouped; unseen values become all zeros.	A new category in production never crashes the API.
Classifier	Logistic Regression (C=1), Random Forest (200 trees), Decision Tree (depth 6).	Simple, explainable models with balanced class weights.

Three models compared

Model	How it thinks	Strength	Weakness
Logistic Regression	Adds up weighted features into a score (log-odds).	Stable, easy to explain, good with little data.	Misses non-linear patterns.
Random Forest	Averages 200 decision trees built on random subsets.	Captures interactions; robust.	Harder to explain; can overfit.

Model	How it thinks	Strength	Weakness
Decision Tree	One tree of yes/no questions.	Very easy to read.	Unstable; usually less accurate.

6. Prediction and explanation flow



Any validation problem stops the request with HTTP 422 and a list of exactly which field failed.

Figure 4 - What happens when POST /api/predict is called

Input rules

- Every model feature must be present as a key. Send **null** if a value is unknown; it will be imputed and a warning is returned.
- Numeric fields accept numbers or numeric text ("38,000"). Text such as "high", booleans or infinity are rejected.
- Values outside the training range are accepted with a warning, because the model is less reliable there.
- Unknown categories are accepted with a warning. Extra fields are ignored with a warning.

Two separate decisions

- **predicted_class** - default if probability is at least the tuned decision threshold.
- **risk_band** - from configurable probability bands. Default: Low below 0.20, Medium below 0.50, High otherwise.

How SHAP explanations work

SHAP starts from a baseline (the average model output) and assigns each borrower field a value showing how much it moved this borrower's score up (more risk) or down (less risk). Base value plus all SHAP values equals the model's output exactly; the tests check this. Tree models use TreeExplainer and report in probability units. Logistic regression uses LinearExplainer and reports in log-odds units. One-hot columns are added back together, so explanations always name the original field, such as home_ownership, not home_ownership==RENT.

SHAP explains how the **model** behaves. It does not prove that a factor **causes** default in the real world.

7. API reference

Base URL when running locally: **http://127.0.0.1:8000**. Interactive docs: **/docs**. Call order: upload, then train, then predict / explain.

Method + path	Purpose	Key input	Key output
GET /api/health	Is the server up?	-	status, dataset_loaded, model_loaded
POST /api/data/upload	Upload and validate a loan CSV	file, target_column, positive_labels, negative_labels, is_synthetic (form-data)	dataset info + quality report (201)
GET /api/data/summary	Dataset statistics and quality	optional target overrides (query)	missing, duplicates, types, class balance, leakage warnings
POST /api/model/train	Train and compare the 3 models	JSON, all optional: models, test_size, selection_metric, risk_bands...	selected model, threshold, ranked comparison, warnings
GET /api/model/metrics	All evaluation results	-	per-model train/val/test metrics, confusion matrices, reports
GET /api/model/status	Is a model ready? Which inputs?	-	model name, feature schema, risk bands, test metrics
POST /api/predict	Score borrowers	records (1-1000), include_explanation, top_k	probability, class, risk band, top factors, warnings
POST /api/explain	SHAP explanations	records (optional), include_global, top_k	global importance, summary plot data, local factors

Example: predict one borrower

```
POST /api/predict
{
  "records": [{
    "age": 29, "annual_income": 38000, "employment_length_years": 1.5,
    "employment_type": "contract", "home_ownership": "RENT", "loan_amount": 25000,
    "loan_term_months": 60, "interest_rate": 21.5, "loan_purpose": "small_business",
    "credit_score": 590, "debt_to_income": 34.0, "credit_utilization_pct": 85.0,
    "num_open_accounts": 12, "delinquencies_2y": 2
  }],
  "include_explanation": true,
  "top_k": 3
}
```

```
200 OK
{
  "model": "logistic_regression", "decision_threshold": 0.553879,
  "trained_on_synthetic_data": true,
  "predictions": [{
    "index": 0, "default_probability": 0.966977,
    "predicted_class": "default", "predicted_label": 1, "risk_band": "High",
    "warnings": [],
    "top_factors": {
      "output_space": "log_odds",
      "increasing_risk": [
        {"feature": "credit_score", "value": 590.0, "shap_value": 1.35973, "effect": "increases_risk"},
        {"feature": "delinquencies_2y", "value": 2.0, "shap_value": 0.525033, "effect": "increases_risk"}
      ],
      "decreasing_risk": [ ... ]
    }
  }],
  "disclaimer": "Predictions are statistical estimates ..."
}
```

Tip for the frontend: call GET /api/model/status first. Its feature_schema lists every input field, its training range and allowed categories, so the borrower form can be built from it automatically.

8. Error handling

Every error, from any endpoint, has the same JSON shape, so the frontend needs only one error handler:

```
{ "error": { "code": "model_not_trained", "message": "No trained model available. Call POST /api/model/train first
```

HTTP	code	Typical cause
409	dataset_not_loaded / model_not_trained / training_in_progress	Endpoints called in the wrong order, or two trainings at once
413	http_413	Uploaded file larger than the limit (20 MB default)
415	http_415	File is not .csv or has a wrong content type
422	request_validation_error	JSON body has wrong fields or types (Pydantic)
422	data_validation_error / insufficient_data	Wrong target column, unclear labels, fewer than 50 rows or fewer than 10 per class
422	invalid_input	Borrower record missing fields or with wrong types; details list each field
500	internal_error	Unexpected bug; details are logged on the server, not exposed

9. Current results

These numbers come from SYNTHETIC development data. They prove the pipeline works end to end, but they say nothing about real-world credit performance. Do not present them as real results.

Trained at **2026-10-22 18:18:36 UTC**. Selected model: **logistic_regression** (best validation roc_auc). Decision threshold: **0.553879**. Split: 1800 train / 600 validation / 600 test rows, default rate about 19.8% in each split.

Model	Val roc_auc	Test precision	Test recall	Test F1	Test ROC-AUC	Train ROC-AUC
logistic_regression (selected)	0.6685	0.3409	0.5042	0.4068	0.6768	0.7371
random_forest	0.6553	0.3407	0.521	0.412	0.6931	0.8919
decision_tree	0.5802	0.2837	0.5126	0.3653	0.6512	0.8179

Confusion matrix of the selected model on the test split:

	Predicted non-default	Predicted default
Actual non-default	365	116
Actual default	59	60

How to read it: of 119 real defaults in the test set, the model caught 60 (recall 0.5042). Of all borrowers it flagged as default, 34% really defaulted (precision 0.3409). Lowering the threshold catches more defaults but flags more good borrowers; the *min_default_recall* training option controls this trade-off.

Columns removed before training: **total_recovered_amount** (suspected post-outcome field (target leakage)), **loan_id** (identifier column).

10. Run it yourself, step by step

All commands are for Windows PowerShell and run from the **backend** folder.

Step 1 - Create the environment and install packages (once)

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Step 2 - (Optional) regenerate the synthetic dataset

```
.\.venv\Scripts\python.exe scripts\generate_synthetic_data.py
```

Step 3 - Start the API server (leave this terminal open)

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Step 4 - Open the interactive docs in a browser

```
http://127.0.0.1:8000/docs
```

Step 5 - Upload the dataset (second terminal)

```
curl.exe -F "file=@data/synthetic_loans.csv" -F "target_column=loan_outcome" `
         -F "positive_labels=Defaulted" http://127.0.0.1:8000/api/data/upload
```

Step 6 - Train the models

```
Invoke-RestMethod -Method Post http://127.0.0.1:8000/api/model/train -ContentType application/json -Body '{}'
```

Step 7 - Check the metrics

```
Invoke-RestMethod http://127.0.0.1:8000/api/model/metrics | ConvertTo-Json -Depth 4
```

Step 8 - Predict and explain

```
Use POST /api/predict and POST /api/explain from /docs with the example body in Section 7.
```

Step 9 - Run all automated tests

```
.\.venv\Scripts\python.exe -m pytest -q
```

Step 10 - Rebuild this PDF after retraining

```
.\.venv\Scripts\python.exe -m pip install -r docs\requirements-docs.txt
.\.venv\Scripts\python.exe docs\build_architecture_pdf.py
```

Using the real dataset

1. Upload the team's CSV with its real target column name and the label(s) that mean default, e.g. target_column=loan_status, positive_labels=Charged Off.
2. If the target has extra values such as Current or Late, also send negative_labels (e.g. Fully Paid).
3. Read the summary: check leakage_warnings and excluded_columns, and add any post-outcome fields to exclude_columns.
4. Train, review /api/model/metrics, then rebuild this PDF so the results section shows the real numbers.

11. Testing

47 automated tests, all passing. Run with **pytest -q** (about 5-15 seconds). Each test uses its own temporary folders, so tests never touch the real model or data.

File	Covers
tests/test_data_processing.py	CSV loading and rejection, target mapping and label inference, ambiguous labels, ID/leakage/constant detection, numbers stored as text, quality report, statistical leakage, too-small or single-class data.
tests/test_model_pipeline.py	Missing-value imputation, categorical encoding, unseen categories, preprocessing fitted on train only, training all models, reproducibility, exact metric values, threshold tuning, probability range 0-1, risk bands, input validation, save/load, SHAP additivity for all 3 models, global importance.
tests/test_api.py	Every endpoint: success responses, calling before data/model exists (409), wrong file type (415), oversized file (413), invalid bodies (422), invalid borrower data (422), model survives a server restart.

12. Limitations and next steps

Limitation	Impact	Suggested next step
No authentication	Anyone who can reach the server can upload, train and predict.	Run only locally, or put it behind the team's auth / API gateway.
Synthetic data only	Current metrics are not real-world evidence.	Upload the real dataset and retrain.
One dataset and one model at a time, file storage	No model history; not built for several server workers.	Add model versioning or a database if the project needs it.
Fixed hyperparameters	Models may not be at their best.	Add cross-validated hyperparameter search.
Probabilities not calibrated	0.30 is a relative score, not an exact 30% chance.	Add probability calibration on validation data.

Limitation	Impact	Suggested next step
Automatic leakage detection is heuristic	Some post-outcome fields could slip through.	Domain review of every column with the business team.
joblib files can run code when loaded	A malicious model file is a security risk.	Only load models produced by this service.

Predictions are statistical estimates based on historical data. They are not guarantees of borrower behaviour and should support, not replace, human credit decisions and applicable lending regulations.